Pokhara University

Gandaki College of Engineering and Science

Lamachaur, Kaski

Lab 5: Implementation of CI/CD Pipelines Using GitHub Actions and Jenkins

Submitted By:

Ashutosh Parajuli

BESE2023

Roll no. 17

Submitted To:

Er. Rajendra Bahadur Thapa

AIM:

To build Continuous Integration and Continuous Deployment (CI/CD) pipelines using GitHub Actions and Jenkins, automating the build, test, and deployment stages of a software project.

THEORY:

CI/CD automates the process of integrating code changes and deploying them, resulting in faster and more dependable software delivery, a principle central to Agile methodology.

- Continuous Integration (CI): builds and tests code automatically on every push or pull request, allowing errors to be caught at an early stage.
- Continuous Deployment (CD): automatically deploys code that has passed CI to a designated target environment.
- GitHub Actions: CI/CD functionality built directly into GitHub, where pipelines are described as YAML files placed inside .github/workflows/ and are triggered by events such as push or pull_request.
- Jenkins: an open-source automation server whose pipelines are described in a Jenkinsfile written using the Groovy DSL, and which is highly extensible through hundreds of available plugins.

OBJECTIVES:

- Build a GitHub Actions workflow that compiles and tests a Node.js project on every push.
- Set up a Jenkins pipeline containing build and test stages.
- Trigger the pipelines and monitor their automated execution.
- Compare GitHub Actions with Jenkins in terms of setup effort and usability.

ALGORITHM:

1. Set up a Node.js project that includes a test script written with Jest.
2. Push the project to a repository hosted on GitHub.
3. Create a .github/workflows/ci.yml file that defines the GitHub Actions pipeline.
4. Push the workflow file and monitor the Actions tab on GitHub.
5. Install Jenkins, either locally or on a server.
6. Set up a new Jenkins Pipeline job.
7. Write a Jenkinsfile and place it in the root of the project.
8. Trigger a Jenkins build and inspect the console output.

CODE:

GitHub Actions – .github/workflows/ci.yml:

```yaml
name: CI Pipeline



on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]


jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
     - uses: actions/checkout@v3
     - uses: actions/setup-node@v3

       with:
         node-version: '18'

     - run: npm install
     - run: npm test
```

Jenkins – Jenkinsfile (Declarative Pipeline):

```
pipeline {
    agent any
    stages {
        stage('Checkout') { steps { git 'https://github.com/user/repo.git' } }
        stage('Install') { steps { sh 'npm install' } }
        stage('Test')     { steps { sh 'npm test'    } }
        stage('Build')    { steps { sh 'npm run build' } }
    }
    post {
        success { echo 'Pipeline completed successfully!' }
        failure { echo 'Pipeline failed.' }
    }
}
```

OUTPUT:

GitHub Actions (Actions tab):

```
✓ Checkout code
✓ Setup Node.js
✓ Install dependencies
✓ Run tests

Workflow completed successfully.
```

Jenkins console:

```
[Pipeline] stage: Install

[Pipeline] stage: Test -> 2 tests passed [Pipeline]

stage: Build

Finished: SUCCESS
```

RESULT:

The CI/CD pipelines were implemented successfully using both GitHub Actions and Jenkins, with the automated build and test stages running correctly whenever code was pushed.

CONCLUSION:

GitHub Actions provides a seamless, zero-infrastructure setup that is tightly integrated with GitHub, whereas Jenkins offers greater flexibility for handling complex enterprise workflows. Both tools are widely adopted in the industry and complement one another depending on the needs of the project.